

Assignment 1 Report: HTML Basics

Student: Arlan Kudabay SE-2537

Topic: Invictus GO Gym (Mangilik El 18, Astana)

Date: September 13, 2026

Repository: <https://github.com/zywko/webasikinvictus>

Task A — Anatomy of a Real Page

Analyzed Website: <https://invictusgo.kz/>

- Technical Parameters in Source Code:
 1. Doctype: `<!DOCTYPE html>` — regular HTML5 doctype.
 2. Lang: `lang="ru"` inside the `<html>` tag.
 3. Charset: `<meta charset="UTF-8">` for text encoding.
 4. Viewport: `<meta name="viewport" content="width=device-width, initial-scale=1.0">` so it scales on mobile screens.
 5. Title: `<title>Сеть фитнес-клубов Invictus Fitness</title>`
 6. Three Meta Tags:
 1. `<meta name="description" content="...">` with club details.
 2. `<meta name="robots" content="index, follow">` for search indexing.
 3. `<meta property="og:title" content="Invictus Fitness">` for social previews.
- Structural Analysis:

The real site looks like it was built with modern frameworks and page builders. Because of that, almost everything is wrapped in endless `<div>` tags. There are no proper `<main>` or `<article>` landmarks, and the layout relies completely on CSS classes rather than semantic HTML.
- Three Structural Mistakes Found and Avoided:
 1. Too many div wrappers: The real site puts simple paragraphs inside 5–6 layers of empty `<div>` tags. In my code, I used clear tags like `<section>` and `<article>`.
 2. Fake pricing tables: They built the membership prices using flexbox divs. It looks fine visually, but screen readers can't read it as a table. In my `memberships.html`, I wrote a real `<table>` with `<thead>`, `<tbody>`, and row/col headers.
 3. Messy headings: On the live site, heading levels jump around randomly (e.g. from `<h1>` straight to `<h4>`). In my project, I kept a strict order: one `<h1>` per page, followed by `<h2>` and `<h3>`.

Hand-Drawn Page Sketch:

<header>

<h1 Invictus GO Astana (Mangilik El 18)></h1>

<p Subtitle / Branch 1100 ></p>

<nav>

[Home] | [Prices & Rules] | [Sign up] | [Contact]

</nav>

Course: Web Tech.
Student: Arlan Kudabayev
Date: September 10

Task B — Written Part

1. How a Web Page Works and Browser Rendering:

An HTML page is just a text file with tags, attributes, and content. When you type an address in the browser, it downloads this text as raw bytes over the internet. The browser reads the bytes, figures out the encoding (UTF-8), and starts turning the text into tags.

Next, it builds the DOM (Document Object Model), which is basically an upside-down tree where each tag is an object connected to its parent and children. Since this project has zero CSS, the browser just uses its default built-in styling rules (User Agent stylesheet). It figures out how big blocks are and where they sit on the screen (layout stage), and then draws the letters, borders, and pictures pixel by pixel (paint stage). The browser does this while reading the file from top to bottom, so parts of the page start showing up even before the whole file finishes downloading.

2. Three Semantic Choices Over `<div>`:

- `index.html` (Line 32) — `<section>`: Instead of an empty `<div>`, I put `<section id="hall-info">` because it's a specific topic about gym gear and lockers with its own `<h2>` title.
- `index.html` (Line 54) — `<article>`: I used `<article>` for the visitor review because it's an independent piece of text that makes sense even if you take it away from the rest of the site.
- `index.html` (Lines 45–48) — `<figure>` and `<figcaption>`: Instead of putting the entrance photo in a `<div>`, I used `<figure>` with `<figcaption>`. This ties the picture and its description together so browser tools and screen readers know they belong to each other.

3. What Happens When You Press Submit Today:

When someone fills out the form on `booking.html` and clicks "Submit Application", the browser first checks if all required fields are filled and if the email/phone formats look right. Since we haven't connected a database or server script yet, the form uses `method="get"`. The browser takes all inputs, sticks them onto the end of the web address as query parameters (like `booking.html?full_name=...`), and reloads the same page. No data is actually saved anywhere.